The Empty Number

How one blank field in a phone's modem silenced Wi-Fi calling, and the eleven bytes that fixed it

Prepared with Claude

November 2026

Contents

Preface 2
1. A phone that could call, but not like this 3
The symptom 3
Why that was strange 3
Why Wi-Fi calling matters 4
2. How a call finds its way 4
Two computers in one phone 4
IMS: the system behind calls 4
Feature tags: "this is what I can do" 4
The question 4
3. The modem's checklist 5
A hidden yes/no 5
What the check looks at 5
The field was empty 5
A detail that explains a lot 6
4. Who is supposed to fill it in 6
The phone that works 6
The answer: the emergency-address setup 6
The phone that was missing the step 6
Checking that nothing else could do it 7
5. How we know 7
Two phones, one test 7
Reading a hidden log 7
Where the two phones diverged 7
The decisive test 8
What it wasn't 8
6. Eleven bytes 8
The fix in one sentence 8
Why "eleven bytes" 8
Why it lasts 8
When it needs redoing 9
What a "native" fix would look like 9
7. Doing it yourself 9
What you need 9

Step 1: switch on the phone's diagnostic port 9
Step 2: run the tool 9
Step 3: reboot and check 10
Undoing it 10
Safety notes 10
8. The emergency-call question 10
What the field really is 10
What an emergency address is 10
What the CAT S22 does for a Wi-Fi emergency call 10
What is and is not known 11
What to do 11
9. What the bug says about phones and carriers 11
Capability versus entitlement 11
Carrier rules live in the modem 11
Silent gates 12
Certification covers more than the radio 12
Diagnostic access as a repair tool 12
Fragility 12
10. What we still don't know 12
Appendix A: Glossary 13
Appendix B: Technical reference 13
The settings file 13
What gets written 14
The decision chain, in modem terms 14
Who writes the field 14
Reading the modem's compact log 14
Other facts 14
Appendix C: Credits and further reading 15

Preface

This is the story of a small bug with a large shadow.

A rugged Android phone, the CAT S22 Flip, could make ordinary calls and LTE "VoLTE" calls on Verizon without complaint. It could connect to Wi-Fi. It could even *register* for Wi-Fi calling, and send text messages and video over Wi-Fi. But the one thing Wi-Fi calling exists to do, carry a **voice call**, never worked. The phone said, in effect, "Wi-Fi calling is on" and then refused every call.

For a long time the cause looked like it could be almost anything: a carrier setting, a radio quirk, a firmware difference, a missing flag. It turned out to be one blank 20-byte text field in a settings file inside the phone's modem. When that field holds the line's own phone number, voice over Wi-Fi works. When it is empty, it silently does not.

This book explains:

- **what the problem was**, in plain language, with just enough background about how phone calls are set up to make it make sense;

- **why it happened**, including why a phone that is perfectly capable of Wi-Fi calling can still be refused by its own modem;
- **how we know**, because "it started working after I changed something" is not the same as understanding;
- **how it is fixed**, and how you could do it yourself;
- and **what it means**, for emergency calls, for phones sold into the wrong carrier's software, and for anyone who has ever stared at a feature that is "on" but does nothing.

You do not need to be an engineer to follow the main chapters. The appendices hold the technical reference for those who want it.

A word on certainty. Where something was measured on the phone, a single change with a clear before and after, this book says it was **proven**. Where it comes from reading the modem's software and was not directly tested, it says **inferred**. The distinction matters, and it is kept throughout.

1. A phone that could call, but not like this

The symptom

The phone is a CAT S22 Flip, a rugged flip phone running Android 11, used on Verizon. Three things worked:

- ordinary calls over the cellular network, including **VoLTE** (voice over LTE, the modern way cellular calls are carried);
- connecting to Wi-Fi and using data over it;
- Wi-Fi calling's *plumbing*: the phone registered with Verizon over Wi-Fi, and text and video worked over that same registration.

One thing did not: **voice over Wi-Fi.** In Android's own diagnostics, the phone reported that it was registered over Wi-Fi with the capabilities:

> Voice: **false** Video: true SMS: true

Outgoing calls over Wi-Fi were refused with the message "Cellular network not available." Incoming calls over Wi-Fi never rang; the phone itself answered the network with a refusal, `486 VoWiFi OFF`, in about thirty-five milliseconds.

Why that was strange

Wi-Fi calling is not exotic. It is a standard feature, and this phone's hardware and modem both support it. The owner had been told that Wi-Fi calling worked on this handset on another carrier. The registration over Wi-Fi was succeeding. Verizon's network was accepting the phone. And yet voice alone was off.

That combination, *everything works except one capability, with no error anywhere*, is the signature of a **gate**: a yes/no check somewhere that quietly says "no" and never explains itself. The hunt was for the gate.

Why Wi-Fi calling matters

It is easy to treat Wi-Fi calling as a convenience. For some people it is the only way to make a call at home or at work where cellular coverage is weak. For a rugged phone used in basements, warehouses and job sites, it can be the difference between a working phone and a brick. That is why a quiet "no" is worth chasing to the bottom.

2. How a call finds its way

To understand the bug you need a small amount of background. None of it is deep.

Two computers in one phone

A smartphone is really two computers sharing a case. One runs Android: the apps, the screen, the settings. The other is the **modem** (also called the baseband), a separate processor with its own software that talks to the cellular network and to Wi-Fi calling servers. Android and the modem exchange messages, but each has its own idea of what is allowed.

This matters because the decision that blocked Wi-Fi voice on this phone was made **inside the modem**, where Android's settings screens cannot see it.

IMS: the system behind calls

Modern phone calls, VoLTE and Wi-Fi calling alike, run on a system called **IMS** (IP Multimedia Subsystem). In IMS a call is just internet traffic set up with a protocol called **SIP**, the same family of protocols used by internet phone systems.

Before you can make or receive calls, your phone **registers** with the carrier: it sends a SIP message named `REGISTER` that says "this is me, this is where to reach me, and this is what I can do."

Feature tags: "this is what I can do"

The "what I can do" part is a list of **feature tags** in the REGISTER message. The one that matters here describes voice:

Tag the phone sends	Meaning
`+g.gsma.rcs.telephony="cs,volte"`	I can do voice calls
`+g.gsma.rcs.telephony="cs"`	I am here, but I am not claiming voice over this connection

If the phone sends `"cs"` only, the network registers it without voice. Android then reads the result and reports **Voice: false**. Nothing is broken in the network, the radio, or Android: the phone simply never *asked* for voice.

The question

On LTE, this phone sent `"cs,volte"`. On Wi-Fi, in every Wi-Fi calling mode, it sent `"cs"`. So the real question was:

> **Why does the modem choose to claim voice on LTE but not on Wi-Fi?**

Answering it took comparing the phone against a twin that did the right thing, and reading the modem's decision logic. The next chapters tell what was found.

3. The modem's checklist

A hidden yes/no

Inside the modem, before it decides to advertise voice on Wi-Fi, a piece of software asks a simple question: **"Is VoWiFi service enabled for this line?"** If the answer is yes, the voice feature tag goes into the REGISTER. If no, it does not.

Everything else in the Wi-Fi calling chain, the tunnel to the carrier, the registration, text, video, can work perfectly while this one answer is "no."

What the check looks at

For a Verizon phone, the check compares two things:

1. a **setting stored in the modem's own storage**, which this book will call the *NV MDN*. ("MDN" is "mobile directory number": the line's own phone number. "NV" means non-volatile: it lives in the modem's permanent storage and survives reboots.)
2. the **SIM card's own phone number**, which the modem reads from the SIM at boot.

If the stored number is present and **matches** the SIM's number, the answer is yes. If it is missing, the answer is no.

The stored number lives in a small configuration file inside the modem, `qp_ims_service_enablement_config`. The relevant part is a 20-byte text field: the line number as plain digits, followed by a zero byte to mark the end.

The field was empty

On this phone, that field was all zeros. The modem therefore had nothing to compare, the check said no, and the chain of consequences followed:

```
empty number in the settings file
        |
        v
"is VoWiFi enabled?" check says NO
        |
        v
voice service inside the modem stays OFF on Wi-Fi
        |
        v
REGISTER over Wi-Fi advertises "cs"  (no voice)
        |
        v
Android reports  Voice: false  and refuses the call
```

This is a satisfying kind of bug: **one small cause, a long and silent chain of effects, and no error message anywhere along it.**

A detail that explains a lot

The modem's check is not the same for every carrier. It depends on an operating mode that tells the modem which carrier's rules to follow. In Verizon's mode, the check needs the stored number to match the SIM. In other modes, including the one used for T-Mobile, the check only needs a simple "Wi-Fi calling is enabled" flag, which Android's ordinary Wi-Fi calling switch already sets.

So the number requirement is **Verizon-specific**. A phone built for another carrier would sail through this check without ever having to supply a number. That is a big clue about how this phone ended up in trouble, and it is the subject of the next chapter.

4. Who is supposed to fill it in

The phone that works

To understand what was missing, the investigation used a second phone that *does* do Wi-Fi calling correctly on Verizon: a Kyocera E4810, an older Android 9 handset built and certified for Verizon. Call it the **reference phone**.

On the reference phone the same field is filled in, and the number it holds equals the SIM's number. The question became: *what puts it there?*

The answer: the emergency-address setup

The reference phone's own software fills the field in as the last step of **Wi-Fi calling's emergency-address setup**. When you turn on Wi-Fi calling on a Verizon phone, you are asked for an address to use for 911. When that setup completes, Kyocera's phone app writes the phone's own line number into the modem:

```
Wi-Fi calling set-up completes (address saved)
   -> phone app: "save this line number"
   -> a vendor-specific hook in the radio software
   -> a message to the modem
   -> the settings file now holds the line number
```

If the address is removed, or the SIM or number changes, the same code writes a zero to clear it. In other words, that field is really a **marker**: *"Wi-Fi calling has been activated for this line."* The modem trusts that marker, and checks it against the SIM, before it will claim voice over Wi-Fi.

The phone that was missing the step

The CAT S22 Flip's software has **none of that**. Its radio software has no such hook. Its Android build contains none of the Kyocera or Verizon activation pieces. And it is a different kind of build altogether: its device group identifies it as a **T-Mobile-flavoured** build, running with a Verizon SIM and Verizon's carrier configuration loaded into the modem.

That is the heart of the matter:

> The phone's modem was running **Verizon's rules**, which demand an activation marker. The phone's Android was a **T-Mobile build**, which has nothing that ever sets one.

Nobody was at fault in the ordinary sense. Each half was correct for the carrier it was built for. Put together, they left a handshake with no one on the other end.

Checking that nothing else could do it

Because "nothing sets it" is a strong claim, it was checked from several directions. In Android's own software, the field corresponds to a setting called `wifi_provisioning_id` in a message sent from Android to the modem. The only Android interface that can set it is a particular vendor service call that **no application on the phone makes**. The radio software's own senders of that message never write the field. Android's standard list of 82 IMS settings has no item for a line number. No carrier-configuration key or system property feeds it. The ROM's own Wi-Fi calling screen only flips standard settings; its "emergency address" button launches an action that nothing on the phone handles.

So it was not that a condition failed. **Nothing ever tried.**

5. How we know

A fix that "seems to work" is easy to get wrong. This chapter explains why the conclusion here is solid.

Two phones, one test

The key was a **like-for-like comparison.** The same test was run on both phones: put the phone on Wi-Fi only, with no cellular connection to lean on, start a detailed capture of everything the modem was logging, and watch what it did when registering for Wi-Fi calling.

The reference phone registered over Wi-Fi with `"cs,volte"` and had voice. The CAT S22 registered with `"cs"` and did not. Same carrier, same kind of network, different decision. That narrowed the question to *what differs inside the modem at the moment it decides.*

Reading a hidden log

The modem logs a great deal, but most of the interesting messages on this phone were stored in a compact binary format the usual tools did not decode. The breakthrough was working out how to read it. Each log entry carries a short fingerprint of its message text, and that fingerprint turned out to be a standard one: the **last four bytes of the MD5 hash of the source file name plus the message text**. Because the same message has the same fingerprint on both phones, the two logs could be lined up side by side even where the wording was unknown, and the places where they *diverged* stood out.

(The technical details are in Appendix B.)

Where the two phones diverged

Lining up the two phones' logs from the moment the Wi-Fi connection came up to the moment the voice tag was chosen, they were nearly identical. They parted ways at exactly two points, and both were the same cause seen twice:

- the modem's "is VoWiFi enabled" check returned **1 (yes)** on the reference phone and **0 (no)** on the CAT S22;
- the voice service inside the modem went from off to **on** on the reference phone, and stayed **off** on the CAT S22.

Reading the modem's code for those two spots showed that both consulted the same configuration record, and that the one input that differed between the phones was the stored line-number string. A live read of the file on the CAT S22 confirmed it: **all zeros.**

The decisive test

Finally, one controlled change. The phone's own line number was written into the empty field, **and nothing else was changed.** After a reboot, the same Wi-Fi-only test gave:

	Before	After writing the number
Stored number	empty	11 digits
"Is VoWiFi enabled?" check	no	**yes**
Voice service in the modem	off	**on**
Voice tag in the Wi-Fi REGISTER	`cs`	**`cs,volte`**
Verizon's reply	registered, no voice	**200 OK, with voice**
Android's report	Voice: false	**Voice: true**
A real call over Wi-Fi	refused	**worked**

That is a clean before and after, with one variable changed, and it is what turns a theory into a diagnosis.

What it wasn't

For completeness, a few reasonable-sounding explanations were checked and ruled out. The phone's idea of "voice over packet service available" was the same as the working phone's. Wi-Fi-only registration *can* carry voice on Verizon; the reference phone does it. And the large collection of modem settings that look relevant, when copied from the working phone, made no difference, because none of them was this one blank field.

6. Eleven bytes

The fix in one sentence

> Write the SIM's own line number, as plain digits, into bytes `0x0e` through `0x21` of the file `qp_ims_service_enablement_config`.

Why "eleven bytes"

The line number is a "1" followed by ten digits: eleven characters. The field is twenty bytes wide, so the eleven digits are stored at the start, and the remaining bytes stay zero, which marks where the text ends. The file is 1,024 bytes long; only those eleven bytes differ from the original.

Why it lasts

The field lives in the modem's own permanent storage, not in Android. That has three consequences, all of which were confirmed:

- it **survives reboots**;
- it **survives removing root**: the phone was later returned to its stock, unrooted software and Wi-Fi calling still worked;
- it was **not disturbed by Android's own Wi-Fi-calling settings messages**, which were observed to leave that field untouched.

When it needs redoing

The stored number must always equal the SIM's own number. So the fix must be repeated if:

- the SIM or the phone number changes;
- the file is reset, for example by a deep reset of the modem's settings;
- the value was entered wrongly.

On the reference phone, the phone's own software handles this automatically: on a SIM change it clears the marker and runs the activation again. The CAT S22 has nothing doing that, so a person must.

What a "native" fix would look like

It would be tidier if the phone could set the field itself. In principle three things could do it: a small privileged program that sends the one missing message after boot, the reference phone's own software, or a ROM that includes a proper Verizon activation. All three need software or privileges this phone does not have. For now, writing the field once is the practical answer.

7. Doing it yourself

This chapter is a plain-language guide. A tool that does the steps safely is available in the project's repository (called *cat-s22-verizon-wfc-fix*); this chapter explains what it does so you can judge it.

What you need

- a Mac or Linux computer, with Python 3 and a small USB library called libusb;
- a USB cable;
- the SIM's phone number (it is usually shown in the phone's settings under About);
- the phone's **diagnostic port** switched on (see below).

Step 1: switch on the phone's diagnostic port

The modem can be talked to over USB through a special channel called the *diag port*. It is off by default. On the CAT S22 Flip it is switched on by dialing **`##3424#`** (labelled DIAG) in the phone's dialer. On a rooted phone it can also be done from a command line. Once it is on, the computer sees a Qualcomm USB device with a diagnostic interface.

Step 2: run the tool

The tool is one script. It does the following, in order, and pauses for you at the important points:

1. looks for the phone's diag port and reads the settings file;
2. tells you whether the line-number field is empty;
3. asks for the line number at a **hidden prompt** (nothing is echoed to the screen, and the number is never passed on the command line);
4. asks you to confirm;
5. **saves a backup** of the original file to the folder you ran it from;
6. writes the file back with only the eleven bytes changed;
7. reads it back to check the modem really holds the new value.

A read-only mode lets you look first without changing anything, and a restore mode puts a backup back.

Step 3: reboot and check

Reboot the phone so the modem reloads the setting. Make sure Wi-Fi calling is switched on in Android's settings, then try an ordinary call over Wi-Fi. If you can, confirm in Android's diagnostics that the Wi-Fi registration now reports **Voice: true**.

Undoing it

The backup file the tool saved holds the original contents of the settings file. Running the tool in restore mode with that file puts everything back. Because the original field was empty, the backup contains no phone number. If the field *was* already set when you started, the backup holds the earlier value, so treat it like any file with a phone number in it.

Safety notes

- The tool changes one setting in the modem's storage. It never touches the phone's boot software, the modem's own program, or any partition.
- It checks that the file looks like the layout it expects, and refuses to write otherwise.
- It is proven on **one** phone. Another phone with the same symptom may have a different cause. Use the read-only mode first.
- Use it only on a line you own.

8. The emergency-call question

This is the part of the story that deserves the most care.

What the field really is

Chapter 4 explained that on a Verizon phone the stored number is a *marker* saying "Wi-Fi calling has been activated," normally written when the **emergency address setup** completes. Writing the marker by hand tells the modem that activation is done. It does **not** perform the activation. In particular:

> **Our fix does not register an emergency address with Verizon, and nothing on this phone can.**

What an emergency address is

When you call 911 over Wi-Fi, the call does not come from a cell tower whose location the carrier knows. Instead, emergency services rely on the address you registered for Wi-Fi calling (and, increasingly, on location data the phone supplies). On the reference phone, saving the address contacts a Verizon server, which stores it against the phone number. By all indications the address belongs to the **line**, not to the handset, but that was not directly tested.

What the CAT S22 does for a Wi-Fi emergency call

From the modem's logs, with the caveat that no real emergency call was ever placed (and none should be):

- the modem builds its own location from the phone's satellite-positioning fix plus a street address looked up by Android;
- it packages that location in the standard emergency format and prefers the cellular (LTE) path for emergency calls;

- Android tries the modern calling path first for an emergency number, and the modem chooses the radio;
- a failed Wi-Fi emergency call is retried over cellular.

After the fix, the modem also announces that it is capable of emergency calls over Wi-Fi; before the fix it did not.

What is and is not known

Situation	Assessment
Cellular coverage available	Emergency calls should go over Verizon's normal LTE emergency path, which the fix does not touch (inferred, not tested).
Wi-Fi only, no cellular	**Unknown.** It depends on whether Verizon accepts a Wi-Fi emergency call using only the phone's own location. Outcomes could range from "works with a rough location" to "fails."
Before the fix	The phone would not have tried Wi-Fi emergency calls at all, so the fix adds a new, untested path rather than removing a tested one.

What to do

1. **Keep an active SIM and cellular coverage.** Do not treat Wi-Fi calling as your emergency path.
2. **Say your location out loud** at the start of any emergency call made over Wi-Fi.
3. **Register a Wi-Fi-calling emergency address with Verizon through a normal channel.** The reference phone's settings can look up and update the address for the same line; another Verizon-certified phone can do the same; so can Verizon's support. Be careful: saving an address *replaces* it for the whole line, and a wrong address sends responders to the wrong place.
4. **Never test Wi-Fi 911 with a real emergency call.**
5. **Redo the fix after any SIM or number change.** A mismatch drops Wi-Fi voice and probably the Wi-Fi emergency capability with it, leaving cellular as the path.

9. What the bug says about phones and carriers

The fix took eleven bytes. The explanation is worth a few more paragraphs, because the same shape of problem is probably out there on other phones.

Capability versus entitlement

The CAT S22 had the **capability** to do Wi-Fi calling: the hardware, the modem software, the connection to Verizon. What it lacked was an **entitlement marker**: a record that this line had been activated for the service. A phone can be perfectly able to do something and still be refused because it cannot *prove* it has been enabled. That distinction is invisible to a user, who sees only a switch that is "on" and a feature that does nothing.

Carrier rules live in the modem

It is natural to think of "the carrier's rules" as living in Android's settings or in the network. A surprising amount lives in the **modem**: which tags to send, which checks to run, which operating

mode applies. Switching the modem into Verizon's mode also switched on Verizon's checks, including this one, even though Android was a build made for a different carrier. The two halves of the phone were each correct on their own and mismatched together.

Silent gates

The most frustrating thing about this bug was that **nothing said no.** No error, no warning, no log line a user could read. The modem simply declined to claim a capability. Software gates like this are cheap to write and expensive to diagnose, because the absence of an action leaves no trace. The way through was not staring at the failure but comparing it to a phone that succeeded.

Certification covers more than the radio

When a phone is "certified" for a carrier, the radio performance test is only part of it. The certification also covers software handshakes like this one, which a phone sold into a different carrier's software image will never perform. That is why putting a carrier's configuration onto a phone built for another carrier can give a phone that **almost** works: nearly every layer lines up, and one invisible handshake does not.

Diagnostic access as a repair tool

The owner could not have found, let alone fixed, this through any menu. It took access to the modem's diagnostic channel, a working comparison phone, and the patience to read an unfamiliar log format. That raises a fair question about repair and interoperability: how far should a person's right to make their own phone work extend into the carrier-specific software inside it? This book does not try to answer that. It only notes the fix is a local setting, uses no exploit, and changes only the owner's own phone. It is sensible to check your carrier's terms, to use it only on a line you own, and to remember that it stands in for a step that is supposed to include an emergency address.

Fragility

Finally, a note of humility: the fix is a **marker** that a person set by hand. It will not repair itself after a SIM change. It has been observed across reboots and the removal of root, but not across the long list of things a phone can experience in its life. Treat it as a good fix with a short maintenance note, not as a permanent cure.

10. What we still don't know

Honest accounting of the open questions:

- **How long it lasts.** The value survived reboots and un-rooting. It has not been watched through SIM changes, carrier-configuration refreshes, or full modem resets.
- **Wi-Fi-only emergency calls.** Whether Verizon accepts them without a registered address is unknown, and it cannot be safely tested.
- **Other phones.** The same symptom on another phone might share this cause, or might not. It has been verified on exactly one model.
- **A hands-off fix.** A small privileged program that sends the missing message after boot could do what the written value does, without a person touching the diagnostic port. The pieces exist; it has not been built or tested.

- **The internal wiring of one step.** How the voice service's state finally selects the voice tag was inferred from reading the modem's software, not directly tested. The result at both ends of that step, the voice tag before and after, was measured.
- **The emergency-address server's current behaviour.** The code that talks to Verizon's address server is several years old and may have changed.

If you meet the same symptom on another phone, the most useful thing you can do is read the field first, change nothing, and report what you found.

Appendix A: Glossary

Baseband / modem The second computer in a phone; handles the cellular network and Wi-Fi calling signalling.

Diag port A diagnostic channel into the modem, available over USB when switched on.

EFS The modem's built-in file system, where its settings files live.

E911 Enhanced 911: location information that emergency services use to find a caller.

Feature tag A label in a SIP REGISTER message announcing a capability, such as voice.

IMS The system modern phone calls (VoLTE and Wi-Fi calling) run on.

MDN Mobile directory number: the line's own phone number.

NV Non-volatile: stored permanently, surviving reboots.

QMI The message protocol Android and the modem use to talk to each other.

REGISTER The SIP message a phone sends to sign in to the carrier's calling system.

ROM The Android software build on a phone.

SIP The protocol used to set up calls over IP networks.

VoLTE Voice over LTE: voice calls carried over the carrier's 4G data network.

VoWiFi / Wi-Fi calling Voice calls carried over a Wi-Fi connection.

Appendix B: Technical reference

The settings file

Path: `/nv/item_files/ims/qp_ims_service_enablement_config`, 1,024 bytes. Offsets are file offsets.

Offset	Meaning
`0x01–0x03`	three enable bytes
`0x04–0x07`	32-bit service flags (`0x1f` on this phone; bits 0, 1 and 4 must be set for the Wi-Fi calling check)
`0x08`	Wi-Fi calling enabled (1)
`0x09–0x0c`	32-bit call-mode preference (3)
`0x0e–0x21`	**The line number: a 20-byte, zero-terminated string of ASCII digits. The field this fix sets.**
`0x22–0x2b`, `0x2c–0x2f`, `0x30`	miscellaneous bytes and one 32-bit value

What gets written

The 11-digit number `1` + ten digits, then zero bytes. The tool rewrites the whole file and changes only those eleven bytes, then reads it back to verify. The value is read as the "NV MDN" by the modem's Wi-Fi calling check, which requires it to be non-empty and equal to the SIM's number in Verizon's operating mode (mode 0).

The decision chain, in modem terms

1. A configuration record is built from the settings file; the string becomes a field of that record.
2. A check in the modem's packet-data manager ("is VoWiFi service enabled") returns 0 when the string is empty.
3. The IMS service manager's voice service derives its `allowedByConfig` decision from the same record and stays off on Wi-Fi.
4. The registration manager therefore omits the voice feature tag from the Wi-Fi REGISTER.

In other operating modes the check consults only the Wi-Fi-calling-enabled flag at file offset `0x08`; mode 5 additionally requires the string to be non-empty; mode 0 (Verizon) requires it to equal the SIM's number.

Who writes the field

The only runtime writer is the QMI IMS-Settings "set service-enable config" request, whose 20-byte string field is named `wifi_provisioning_id` on the Android side. The modem copies that string into file offset `0x0e` only if the request marks it present. On the reference phone it is sent by Kyocera's phone app through an OEM hook in a patched radio daemon when the Wi-Fi calling emergency-address setup completes, and cleared on removal or SIM change. The only Android interface on the CAT S22′s build that could send it, a call into the vendor IMS "rcsconfig" service with the `WIFI_PROVISIONING_ID` key, is not made by any application on that phone.

Reading the modem's compact log

Many modem log entries are stored as binary frames:

```
0x92 | type | nargs | drop | timestamp[8] | wirehash[4] | arg0 | arg1 | args...
```

- `wirehash` = (subsystem id << 16) | source line number
- `arg0` = 4
- `arg1` = the **last four bytes of MD5(":")**, big-endian
- the remaining values are the message's arguments

Identical messages have identical `arg1` values on different phones, so two captures can be compared frame by frame even without the message text.

Other facts

- Phone: CAT S22 Flip, Android 11, modem build JO.3.3, device group `Cat-NSDS-S22-TMO-030`.
- Reference phone: Kyocera E4810, Android 9, Verizon-certified.
- Protocol used by the tool: HDLC-framed Qualcomm diagnostic commands, EFS2 file operations (hello, get, put) over the USB diag interface (class `ff/ff/30`).

Appendix C: Credits and further reading

The generic machinery for reading and writing a Qualcomm modem's files over its diagnostic port has been explored by others, and was a useful reference:

- EfsTools: https://github.com/JohnBel/EfsTools
- QCSuper: https://github.com/P1sec/QCSuper
- qcom-efs-browser: https://github.com/catxeon/qcom-efs-browser
- edl: https://github.com/bkerler/edl

None of these targets this particular setting, and no code was copied from them.

The tool described in Chapter 7 is licensed under the GNU General Public License, version 3 or later.

This book describes one investigation on one phone. It is not legal advice, not a guarantee, and not a substitute for carrier support. If you have any doubt about emergency calling, keep cellular service active and register your emergency address through your carrier.